



Artisanal groupby

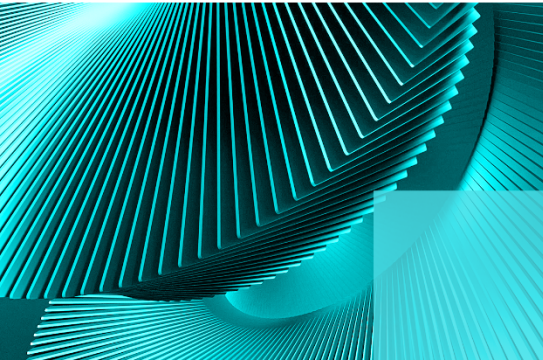
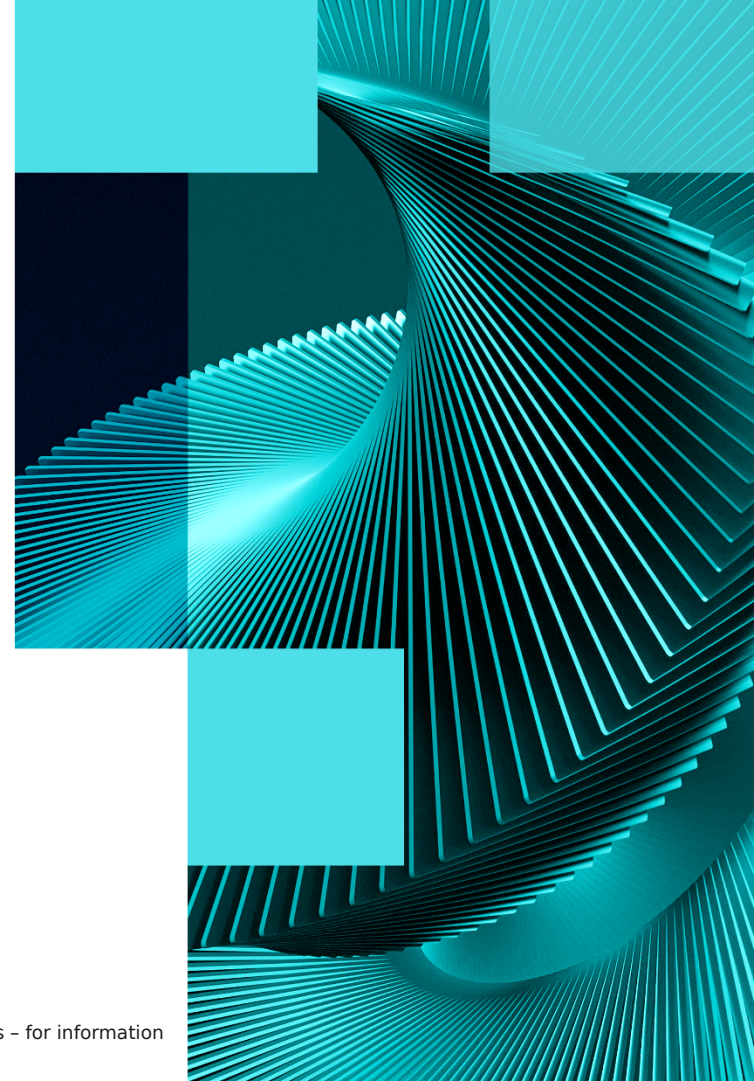
Re-invent the wheel, so it's rounder

George Slavov
Staff Engineer, Man Group

29 April, 2026

Only for C++ Society, Bulgaria. Confidential - not for
onward distribution. Provided for educational purposes.

All extracts provided in this presentation are illustrative examples – for information
only



Agenda

Click

to
add

Problem Statement

Click
to
add

Naïve Implementation

Click
to
add

Exploiting Problem Structure

Click
to
add

Dispatching

Click
to
add

Cost of Memory Allocation

Click
to
add

Click
to
add

Click
to
add

Click
to
add

Click
to
add

Click
to
add

Text

Problem Statement

Pandas primer

```
score
date      security_id msci_ctry
2020-01-01 0          CN
0.180172
0.842409
0.174870
0.160029
0.663623
...
.
2023-01-24 4999      GB
0.523512
```

..

Problem Statement

Pandas primer

```
score
date      security_id msci_ctry
2020-01-01 0          CN
0.180172

          FR
0.842409

          GB
0.174870

          HK
0.160029

          ID
0.663623
...
.
2023-01-24 4999        GB
0.523512
```

Want:

```
.groupby(["date",
"msci_ctry"]).mean()
```

```
date      msci_ctry      score
2020-01-01 CN          0.506151
          FR          0.501836
          GB          0.493498
          HK          0.496232
          ID          0.504691
...
2023-01-24 GB          0.502062
          HK          0.494458
          ID          0.506234
          TW          0.504002
          US          0.501035
```

```
[5600 rows x 3 columns]
```

Problem Statement

Pandas primer

```
score
date      security_id msci_etry
2020-01-01 0          CN
0.180172
          FR
0.842409
          GB
0.174870
          HK
0.160029
          ID
0.663623
...
.
2023-01-24 4999      GB
0.523512
          UK
```

Representation

```
Levels=['2020-01-01', '2020-01-02', ...
        '2023-01-23', '2023-01-24'],
        dtype='datetime64[ns]',
        name='date',
        length=800, freq='B')

Codes =[0, 0, ..., 799, 799],
        dtype=int16,
        length=28000000

..
= [0] * 35000
+ [1] * 35000
...
+ [799] * 35000
```

Problem Statement

Pandas primer

```
score
date      security_id msci_ctry
2020-01-01 0          CN
0.180172
0.842409
0.174870
0.160029
0.663623
...
2023-01-24 4999      GB
0.523512
```

Representation

```
Levels=['CN', 'FR', 'GB', 'HK',
        'ID',
        'TW', 'US'],
        dtype='object',
name='msci_ctry'

Codes =[0, 1, 2, ..., 4, 5, 6],
        dtype=int8

        = [0, 1, 2, 3, 4, 5, 6] *
4000000
```

..

Naïve implementation

Agg interface

```
class mean_aggregator {  
public:  
    void add(double x) {  
        sum += x;  
        ++count;  
    }  
    double get() { return sum /  
static_cast<double>(count); }  
  
private:  
    double sum = 0;  
    std::int64_t count = 0;  
};
```

Naïve implementation

Algorithm

```
template <typename aggregator>
std::tuple<std::vector<double>, std::vector<std::int64_t>>
group_by(const std::vector<double>& data, const std::vector<std::int64_t>&
groups) {
    google::dense_hash_map<std::int64_t, aggregator> aggregators;
    aggregators.set_empty_key(std::numeric_limits<std::int64_t>::min());
    for (size_t i = 0; i < groups.size(); ++i) {
        aggregators[groups[i]].add(data[i]);
    }

    // Uses aggregator.get()
    return extract_results(aggregators);
}
```

Quick benchmark

Data: 28 million doubles

Group: [0, 1, 2, ..., 5, 6] repeated 4 million times

Implementation	Time (milliseconds)
Pandas	623ms
Naïve	104ms
Mine	101ms

Composite group key

Algorithm

```
template <typename aggregator>
std::tuple<std::vector<double>, std::vector<std::vector<std::int64_t>>>
group_by(const std::vector<double>& data,
        const std::vector<std::vector<std::int64_t>>& groups) {
    google::dense_hash_map<std::vector<std::int64_t>, aggregator,
                          boost::hash<std::vector<std::int64_t>>>
aggregators;
    aggregators.set_empty_key(std::vector<std::int64_t>{});
    std::vector<std::int64_t> group_key(groups.size());
    for (size_t i = 0; i < data.size(); ++i) {
        for (size_t j = 0; j < groups.size(); ++j)
            group_key[j] = groups[j][i];
        aggregators[group_key].add(data[i]);
    }

    // Uses aggregator.get()
    return extract_results(aggregators);
}
```

Quick benchmark two groups

Data: 28 million doubles

Group1: [0, 1, 2, ..., 2999] repeated 35000 times

Group2: [0, 1, 2, ..., 5, 6] repeated 4 million times

Implementation	Time (milliseconds)
Pandas	2260ms (3.6x single group)
Naïve	696ms (6.7x)
Mine	111ms (1.1x)

Exploiting Problem Structure

Sorted group

date	score
2020-01-01	0.594461
2020-01-01	0.434986
2020-01-01	0.408266
2020-01-01	0.092581
2020-01-01	0.837539
2020-01-01	0.482095
2020-01-01	0.774148
2020-01-01	0.579926
2020-01-02	0.266455
2020-01-02	0.403292
2020-01-02	0.894127
2020-01-02	0.973578
2020-01-02	0.029932
2020-01-02	0.651206
2020-01-02	0.541361
2020-01-02	0.606755

1. Construct breaks: [0, 8, 16]
2. `Result[i] = mean(`
`score[breaks[i]]:score[breaks[i+`
`1]]`
`)`

Exploiting Problem Structure

Multiple sorted groups

score	date	msci_ctry
0.837539	2020-01-01	CN
0.482095		CN
0.408266		GB
0.092581		GB
0.774148		ID
0.579926		ID
0.594461		US
0.434986		US
0.000000	2020-01-01	CN

1. Construct breaks: [0, 2, 4, ..., 14, 16]
2. `Result[i] = mean(score[breaks[i]]:score[breaks[i+1]])`

Exploiting Problem Structure

Partially sorted group

		score
2020-01-01	US	0.594461
	US	0.434986
	GB	0.408266
	GB	0.092581
	CN	0.837539
	CN	0.482095
	ID	0.774148
	ID	0.579926
2020-01-02	US	0.266455
	US	0.403292
	GB	0.894127
	GB	0.973578
	CN	0.029932
	CN	0.651206
	ID	0.541361
	ID	0.606755

1. Construct breaks: [0, 2, 4, 6, ..., 14, 16]
2. `group_by(`
`score[breaks[i]:breaks[i+1]],`
`msci_ctry[breaks[i]:breaks[i+1]`
`]`
`)`
3. concatenate

Exploiting Problem Structure

Categorical groups

		score
2020-01-01	US	0.594461
	US	0.434986
	GB	0.408266
	GB	0.092581
	CN	0.837539
	CN	0.482095
	ID	0.774148
	ID	0.579926
2020-01-02	US	0.266455
	US	0.403292
	GB	0.894127
	GB	0.973578
	CN	0.029932
	CN	0.651206
	ID	0.541361
	ID	0.606755

Grouping by

```
[ 'US', 'US', 'GB', 'GB',
  'CN', ...
  'CN', 'CN', 'ID', 'ID' ]
```

is equivalent to grouping by

```
[3, 3, 1, 1, 0, ...
 0, 0, 2, 2]
```

Exploiting Problem Structure

Composite categorical groups

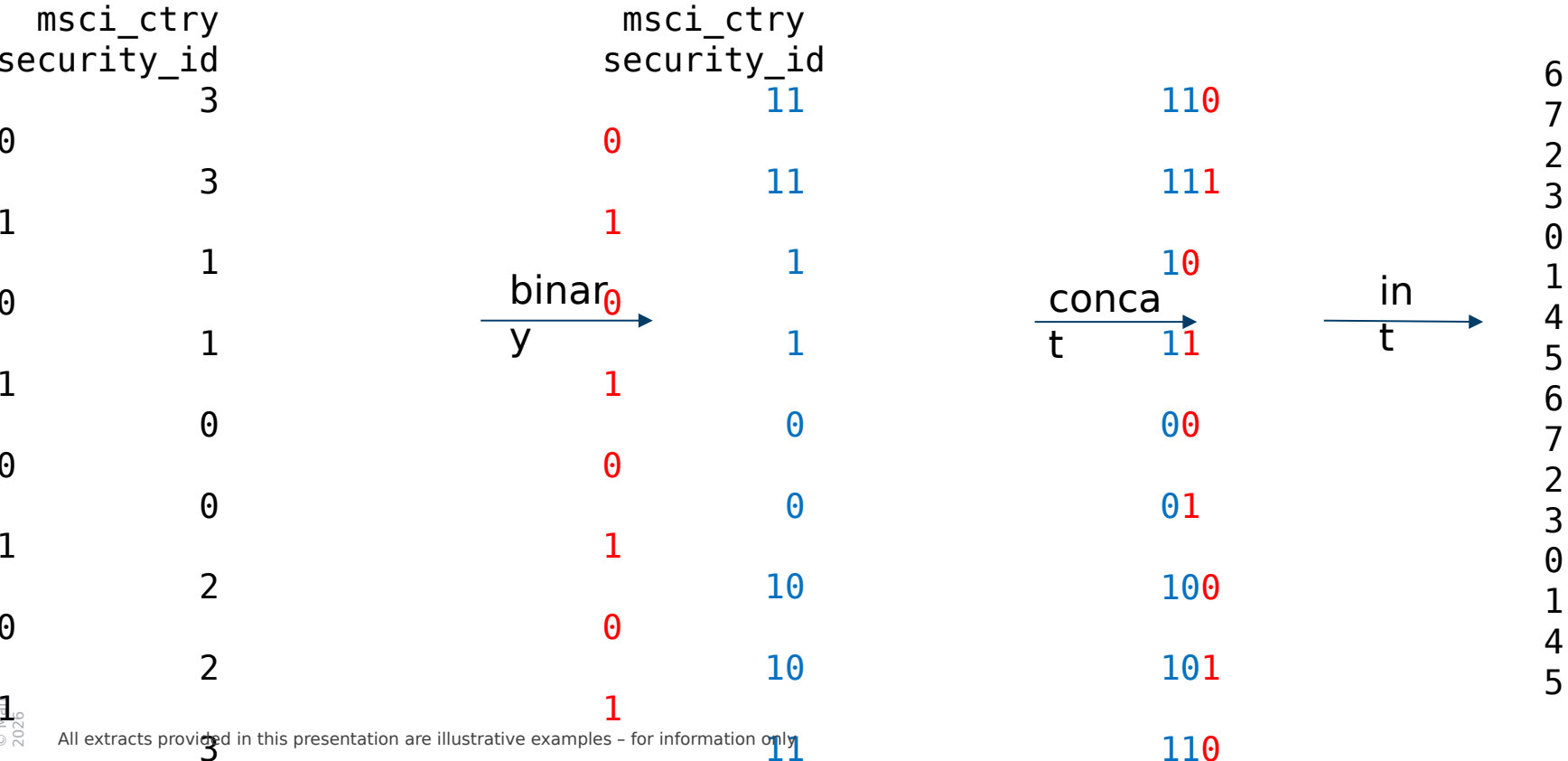
		score		
msci_ctr	security_id		msci_ctr	security_id
US	0	0.594461	0	3
	1	0.434986		3
GB	0	0.408266	1	1
	1	0.092581		3
CN	0	0.837539	0	1
	1	0.482095		1
ID	0	0.774148	1	0
	1	0.579926		1
US	0	0.266455	0	0
	1	0.403292		0
GB	0	0.894127	1	2
	1	0.973578		2
CN	0	0.029932	0	2
	1	0.651206		2
ID	0	0.541361	1	1
	1	0.606755		1

Representation

n

Exploiting Problem Structure

Fusing codes



Dispatching cost

Fusing codes

```
vector<int64_t> construct_unified_grouping(
    size_t num_rows,
    span<variant<span<int8>, span<int16>, span<int32>, span<int64>>>
groups,
    span<int> group_bit_widths) {
    vector<int64> fused(num_rows); // init to 0
    for (size_t i = 0; i < num_rows; ++i) {
        for (size_t j = 0; j < groups.size(); ++j) {
            int shift_amount = group_bit_widths[j];
            fused[i] |= groups[j].visit([shift_amount](auto id) {
                return static_cast<int64_t>(id) << shift_amount;
            });
        }
    }
    return fused;
}
```

Dispatching cost

Fusing codes

```
vector<int64_t> construct_unified_grouping(
    size_t num_rows,
    span<variant<span<int8>, span<int16>, span<int32>, span<int64>>>
groups,
    span<int> group_bit_widths) {
    vector<int64> fused(num_rows); // init to 0
    for (size_t i = 0; i < num_rows; i += 1024) {
        for (size_t j = 0; j < groups.size(); ++j) {
            int shift_amount = group_bit_widths[j];
            groups[j].visit([&fused, shift_amount, i](auto id) {
                for (size_t k = 0; k < 1024; ++k) {
                    fused[i + k] |=
                        static_cast<int64_t>(id) << shift_amount;
                }
            });
        }
    }
    return fused;
}
```

Dispatching cost

Fusing codes

```
vector<int64_t> construct_unified_grouping(
    size_t num_rows,
    span<variant<span<int8>, span<int16>, span<int32>, span<int64>>>
groups,
    span<int> group_bit_widths) {
    vector<int64> fused(num_rows); // init to 0
    for (size_t i = 0; i < num_rows; i += 1024) {
        for (size_t j = 0; j < groups.size(); ++j) {
            int shift_amount = group_bit_widths[j];
            groups[j].visit([&fused, shift_amount, i](auto id) {
                for (size_t k = 0; k < 1024; ++k) { // BUG?
                    fused[i + k] |=
                        static_cast<int64_t>(id) << shift_amount;
                }
            });
        }
    }
    return fused;
}
```

Cost of Allocating Memory

Code

```
void fill_new_vector(benchmark::State& s) {  
    size_t num_elements = 10'000'000;  
    while (s.KeepRunning()) {  
        std::vector<double> data(num_elements, 3.14);  
        benchmark::DoNotOptimize(data);  
    }  
}
```

```
void fill_existing_vector(benchmark::State& s) {  
    size_t num_elements = 10'000'000;  
    std::vector<double> data(num_elements);  
    while (s.KeepRunning()) {  
        std::fill(data.begin(), data.end(), 3.14);  
    }  
}
```

Cost of Allocating Memory

Benchmark

```

-----
--
Benchmark                                Time    UserCounters...
-----
--
fill_new_vector          58.997498 ms    bytes_per_second=1.33304G/s
fill_existing_vector     16.861981 ms    bytes_per_second=4.97601G/s

```

Cost of Allocating Memory

Profile existing vector

Overhead	Command	Shared Object	Symbol
82.81%	group_by_benchm	group_by_benchmarks	[.] fill_existing_vector()
2.22%	group_by_benchm	[kernel.kallsyms]	[k] do_user_addr_fault
2.20%	group_by_benchm	libc.so.6	[.]
__memset_avx2_unaligned_erms			
1.60%	group_by_benchm	[kernel.kallsyms]	[k] clear_page_erms
1.38%	group_by_benchm	[kernel.kallsyms]	[k]
_raw_spin_unlock_irqrestore			
1.33%	group_by_benchm	[kernel.kallsyms]	[k] mem_cgroup_commit_charge
0.99%	group_by_benchm	[kernel.kallsyms]	[k] try_charge_memcg
0.63%	group_by_benchm	[kernel.kallsyms]	[k] get_mem_cgroup_from_mm
0.53%	group_by_benchm	[kernel.kallsyms]	[k] __rcu_read_unlock
0.53%	group_by_benchm	[kernel.kallsyms]	[k] zap_pte_range

Cost of Allocating Memory

Profile new vector

Overhead	Command	Shared Object	Symbol
13.90%	group_by_benchm	[kernel.kallsyms]	[k] do_user_addr_fault
11.56%	group_by_benchm	[kernel.kallsyms]	[k] clear_page_erms
9.56%	group_by_benchm	group_by_benchmarks	[.] fill_new_vector()
9.38%	group_by_benchm	[kernel.kallsyms]	[k]
mem_cgroup_commit_charge			
7.44%	group_by_benchm	[kernel.kallsyms]	[k] try_charge_memcg
6.40%	group_by_benchm	[kernel.kallsyms]	[k]
_raw_spin_unlock_irqrestore			
3.69%	group_by_benchm	[kernel.kallsyms]	[k] get_mem_cgroup_from_mm
3.59%	group_by_benchm	[kernel.kallsyms]	[k] __rcu_read_unlock
3.26%	group_by_benchm	[kernel.kallsyms]	[k] zap_pte_range
3.14%	group_by_benchm	[kernel.kallsyms]	[k]
folio_prep_large_rmappable			
2.77%	group_by_benchm	[kernel.kallsyms]	[k] __rcu_read_lock
1.37%	group_by_benchm	[kernel.kallsyms]	[k] __handle_mm_fault
1.24%	group_by_benchm	[kernel.kallsyms]	[k] mas_walk
1.08%	group_by_benchm	[kernel.kallsyms]	[k] do_anonymous_page
1.04%	group_by_benchm	[kernel.kallsyms]	[k] blk_cgroup_congested
1.03%	group_by_benchm	[kernel.kallsyms]	[k] consume_stack

Template instantiations

Combinatorial explosion

Aggregators

- Count
- Sum
- Mean
- Stdev
- Min/Max
- First/Last
- Median
- Quantile
- NumUnique

Inputs

- Int8
- Int16
- Int32
- Int64
- Float
- Double

Grouping

- Int8
- Int16
- Int32
- Int64

504
combinations

Benchmarks



=====		
=====		
mean aggregation, float64		
=====		
=====		
Sorted (date)	pandas=2.9360s	prospect=0.0659s
44.6x		
Double sorted (date+country)	pandas=10.6215s	prospect=0.0784s
135.5x		
Unsorted (country)	pandas=8.0428s	prospect=0.6434s
12.5x		
Partially sorted (date+security)	pandas=10.6512s	prospect=0.7718s
13.8x		
Partially sorted (date+country)	pandas=11.6358s	prospect=0.4240s
27.4x		
Fully unsorted (country+security)	pandas=12.2233s	prospect=1.4894s
8.2x		
=====		
=====		

Thank
you!

Important Information

This presentation is provided for educational purposes only.



This information is communicated and/or distributed by the relevant Man entity identified below (collectively the "Company") subject to the following conditions and restriction in their respective jurisdictions.

Opinions expressed are those of the author and may not be shared by all personnel of Man Group plc ('Man'). These opinions are subject to change without notice, are for information purposes only and do not constitute an offer or invitation to make an investment in any financial instrument or in any product to which the Company and/or its affiliates provides investment advisory or any other financial services. Any organisations, financial instrument or products described in this material are mentioned for reference purposes only which should not be considered a recommendation for their purchase or sale. Neither the Company nor the authors shall be liable to any person for any action taken on the basis of the information provided. Some statements contained in this material concerning goals, strategies, outlook or other non-historical matters may be forward-looking statements and are based on current indicators and expectations. These forward-looking statements speak only as of the date on which they are made, and the Company undertakes no obligation to update or revise any forward-looking statements. These forward-looking statements are subject to risks and uncertainties that may cause actual results to differ materially from those contained in the statements. The Company and/or its affiliates may or may not have a position in any financial instrument mentioned and may or may not be actively trading in any such securities. Unless stated otherwise all information is provided by the Company. Past performance is not indicative of future results.

Unless stated otherwise this information is communicated by the relevant entity listed below.

Australia: To the extent this material is distributed in Australia it is communicated by Man Investments Australia Limited ABN 47 002 747 480 AFSL 240581, which is regulated by the Australian Securities & Investments Commission ('ASIC'). This information has been prepared without taking into account anyone's objectives, financial situation or needs.

Austria/Germany/Liechtenstein: To the extent this material is distributed in Austria, Germany and/or Liechtenstein it is communicated by Man (Europe) AG, which is authorised and regulated by the Liechtenstein Financial Market Authority (FMA). Man (Europe) AG is registered in the Principality of Liechtenstein no. FL-0002.420.371-2. Man (Europe) AG is an associated participant in the investor compensation scheme, which is operated by the Deposit Guarantee and Investor Compensation Foundation PCC (FL-0002.039.614-1) and corresponds with EU law. Further information is available on the Foundation's website under www.eas-liechtenstein.li.

European Economic Area: Unless indicated otherwise this material is communicated in the European Economic Area by Man Asset Management (Ireland) Limited ('MAMIL') which is registered in Ireland under company number 250493 and has its registered office at 70 Sir John Rogerson's Quay, Grand Canal Dock, Dublin 2, Ireland. MAMIL is authorised and regulated by the Central Bank of Ireland under number C22513.

Hong Kong SAR: To the extent this material is distributed in Hong Kong SAR, this material is communicated by Man Investments (Hong Kong) Limited and has not been reviewed by the Securities and Futures Commission in Hong Kong.

Japan: To the extent this material is distributed in Japan it is communicated by Man Group Japan Limited, Financial Instruments Business Operator, Director of Kanto Local Finance Bureau (Financial Instruments firms) No. 624 for the purpose of providing information on investment strategies, investment services, etc. provided by Man Group, and is not a disclosure document based on laws and regulations. This material can only be communicated only to professional investors (i.e. specific investors or institutional investors as defined under Financial Instruments Exchange Law) who may have sufficient knowledge and experience of related risks.

Switzerland: To the extent this material is made available in Switzerland the communicating entity is Man Investments AG, Huobstrasse 3, 8808 Pfäffikon SZ, Switzerland, which is regulated by the Swiss Financial Market Supervisory Authority ('FINMA').

United Kingdom: Unless indicated otherwise this material is communicated in the United Kingdom by Man Solutions Limited ('MSL') which is a private limited company registered in England and Wales under number 3385362. MSL is authorised and regulated by the UK Financial Conduct Authority (the 'FCA') under number 185637 and has its registered office at Riverbank House, 2 Swan Lane, London, EC4R 3AD, United Kingdom.

United States: To the extent this material is distributed in the United States, it is communicated and distributed by Man Investments, Inc. ('Man Investments'). Man Investments is registered as a broker-dealer with the SEC and is a member of the Financial Industry Regulatory Authority ('FINRA'). Man Investments is also a member of the Securities Investor Protection Corporation ('SIPC'). Man Investments is a wholly owned subsidiary of Man Group plc. The registration and memberships described above in no way imply a certain level of skill or expertise or that the SEC, FINRA or the SIPC have endorsed Man Investments. Man Investments Inc, 1345 Avenue of the Americas, 21st Floor, New York, NY 10105

This material is proprietary information and may not be reproduced or otherwise disseminated in whole or in part without prior written consent. Any data services and information available from public sources used in the creation of this material are believed to be reliable. However accuracy is not warranted or guaranteed. © Man 2026